Relational Algebra and dplyr Verbs

Dom Richards

2025-10-05

1 Overview

We'll now look at the dplyr verbs corresponding to each operator in Codd's relational algebra. As a running example, we'll use the nycflights13 dataset. This dataset contains information about all flights that departed from New York City (e.g. EWR, JFK and LGA) to destinations in the United States, Puerto Rico, and the American Virgin Islands in 2013: 336,776 flights in total. To help understand what causes delays, it also includes a number of other useful datasets:

- `flights`: all flights that departed from NYC in 2013
- `weather`: hourly meterological data for each airport
- `planes`: construction information about each plane
- `airports`: airport names and locations
- `airlines`: translation between two letter carrier codes and names

1.1 Starting a Spark Session

First we need to start a Spark session in local mode. We also ensure the nycflights13 package is installed and loaded.

```
library(sparklyr)
library(dplyr)

spark_disconnect_all()
```

```
## [1] 0
```

```
conf <- spark_config()
conf[["spark.env.SPARK_LOCAL_IP.local"]] <- "0.0.0.0"
conf[["spark.driver.host"]] <- "127.0.0.1"
conf[["spark.ui.port"]] <- 4040

sc <- spark_connect(
  master = "local[2]",
  spark_home = "/opt/spark/spark-3.4.0-bin-hadoop3",
  version = "3.4.0",
  hadoop_version = "3",
  config = conf
)

spark_connection_is_open(sc)
```

```
## [1] TRUE
```

```
if (!"nycflights13" %in% rownames(installed.packages())) {
  install.packages("nycflights13")
}
library(nycflights13)
```

We can now load the data from our local R session into Spark. We'll use the `sparklyr::copy_to()` function, which copies data frames into Spark tables on our spark session:

```
flights_tbl <- sparklyr::copy_to(sc, nycflights13::flights, "flights", overwrite = TRUE)
airlines_tbl <- sparklyr::copy_to(sc, nycflights13::airlines, "airlines", overwrite = TRUE)
airports_tbl <- sparklyr::copy_to(sc, nycflights13::airports, "airports", overwrite = TRUE)
planes_tbl <- sparklyr::copy_to(sc, nycflights13::planes, "planes", overwrite = TRUE)
weather_tbl <- sparklyr::copy_to(sc, nycflights13::weather, "weather", overwrite = TRUE)
```

2 Selection Verbs (σ)

The relational algebra selection operator σ corresponds to the `filter()` verb in dplyr. It selects rows that satisfy a given predicate (similar to the filter function on RDDs):

```
flights_tbl %>%
  filter(origin == "JFK", month == 1, day == 1) %>%
  head() %>%
  collect()
```

```
## # A tibble: 6 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
## 1  2013     1     1      542            540         2      923            850
## 2  2013     1     1      544            545        -1     1004           1022
## 3  2013     1     1      557            600        -3      838            846
## 4  2013     1     1      558            600        -2      849            851
## 5  2013     1     1      558            600        -2      853            856
## 6  2013     1     1      558            600        -2      924            917
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dttm>
```

All dplyr operations on a Spark DataFrame are translated into SQL queries that Spark can execute. We can view the SQL generated for a dplyr verb using `show_query()` as follows:

```
flights_tbl %>%
  filter(origin == "JFK", month == 1, day == 1) %>%
  show_query()
```

```
## <SQL>
## SELECT *
## FROM `flights`
## WHERE (`origin` = 'JFK') AND (`month` = 1.0) AND (`day` = 1.0)
```

3 Projection Verbs (π)

The relational algebra projection operator π corresponds to the `select()` verb in dplyr (unfortunate naming, I know...).

```
flights_tbl %>%
  select(year, month, day, carrier, flight, tailnum, dep_delay, arr_delay) %>%
  head() %>%
  collect()
```

```
## # A tibble: 6 x 8
##    year month   day carrier flight tailnum dep_delay arr_delay
##   <int> <int> <int> <chr>    <int> <chr>       <dbl>     <dbl>
## 1  2013     1     1 UA        1545 N14228          2        11
## 2  2013     1     1 UA        1714 N24211          4        20
## 3  2013     1     1 AA        1141 N619AA          2        33
## 4  2013     1     1 B6         725 N804JB         -1       -18
## 5  2013     1     1 DL         461 N668DN         -6       -25
## 6  2013     1     1 UA        1696 N39463         -4        12
```

We can also see the SQL generated by dbplyr for the select operation:

```
flights_tbl %>%
  select(year, month, day, carrier, flight, tailnum, dep_delay, arr_delay) %>%
  show_query()
```

```
## <SQL>
## SELECT
##   `year`,
##   `month`,
##   `day`,
##   `carrier`,
##   `flight`,
##   `tailnum`,
##   `dep_delay`,
##   `arr_delay`
## FROM `flights`
```

4 Union Verbs (U)

The relational algebra union operator U corresponds to the `union()` verb in dplyr:

```
jfk_jan <- flights_tbl %>%
  filter(origin == "JFK", month == 1) %>%
  select(year, month, day, flight)

jfk_feb <- flights_tbl %>%
  filter(origin == "JFK", month == 2) %>%
  select(year, month, day, flight)
```

```
jfk_union <- union(jfk_jan, jfk_feb)
jfk_union %>%
  count() %>%
  collect()
```

```
## # A tibble: 1 x 1
##       n
##   <dbl>
## 1 16368
```

```
union(jfk_jan, jfk_feb) %>%
  show_query()
```

```
## <SQL>
## SELECT `year`, `month`, `day`, `flight`
## FROM `flights`
## WHERE (`origin` = 'JFK') AND (`month` = 1.0)
##
## UNION
##
## SELECT `year`, `month`, `day`, `flight`
## FROM `flights`
## WHERE (`origin` = 'JFK') AND (`month` = 2.0)
```

You can also use `union_all()` to keep duplicates:

```
union_all(jfk_jan, jfk_feb) %>%
  count() %>%
  collect()
```

```
## # A tibble: 1 x 1
##       n
##   <dbl>
## 1 17582
```

5 Difference Verbs (-)

Set difference corresponds to the `setdiff()` verb in dplyr:

```
jan_all <- flights_tbl %>%
  filter(month == 1) %>%
  select(year, month, day, flight)

feb_all <- flights_tbl %>%
  filter(month == 2) %>%
  select(year, month, day, flight)

only_in_jan <- setdiff(jan_all, feb_all)
only_in_jan %>%
  count() %>%
  collect()
```

```
## # A tibble: 1 x 1
##       n
##   <dbl>
## 1 24201
```

```
setdiff(jan_all, feb_all) %>%
  show_query()
```

```
## <SQL>
## (
##   SELECT `year`, `month`, `day`, `flight`
##   FROM `flights`
##   WHERE (`month` = 1.0)
## )
## EXCEPT
## (
##   SELECT `year`, `month`, `day`, `flight`
##   FROM `flights`
##   WHERE (`month` = 2.0)
## )
```

6 Join Verbs (⋈)

The inner join operator ⋈ corresponds to the `inner_join()` verb in dplyr:

```
joined <- flights_tbl %>%
  inner_join(airlines_tbl, by = "carrier") %>%
  select(year:day, carrier, name, flight, tailnum)

joined %>%
  head() %>%
  collect()
```

```
## # A tibble: 6 x 7
##    year month   day carrier name           flight tailnum
##   <int> <int> <int> <chr>   <chr>           <int> <chr>
## 1  2013     1     1 VX      Virgin America    399 N627VA
## 2  2013     1     1 VX      Virgin America     11 N635VA
## 3  2013     1     1 VX      Virgin America    407 N846VA
## 4  2013     1     1 VX      Virgin America    251 N641VA
## 5  2013     1     1 VX      Virgin America     23 N625VA
## 6  2013     1     1 VX      Virgin America    409 N839VA
```

```
joined %>%
  show_query()
```

```
## <SQL>
## SELECT
##   `year`,
##   `month`,
```

```
##   `day`,
##   `flights`.`carrier` AS `carrier`,
##   `name`,
##   `flight`,
##   `tailnum`
## FROM `flights`
## INNER JOIN `airlines`
##   ON (`flights`.`carrier` = `airlines`.`carrier`)
```

7 Rename Verbs (ρ)

We can rename columns using `rename()`, which corresponds to the relational algebra rename operator ρ:

```
flights_tbl %>%
  rename(departure_delay = dep_delay, arrival_delay = arr_delay) %>%
  select(year:day, departure_delay, arrival_delay) %>%
  head() %>%
  collect()
```

```
## # A tibble: 6 x 5
##    year month   day departure_delay arrival_delay
##   <int> <int> <int>           <dbl>         <dbl>
## 1  2013     1     1               2            11
## 2  2013     1     1               4            20
## 3  2013     1     1               2            33
## 4  2013     1     1              -1           -18
## 5  2013     1     1              -6           -25
## 6  2013     1     1              -4            12
```

8 Aggregation Verbs (γ)

The relational algebra aggregation operator γ corresponds to `group_by()` combined with `summarize()` in dplyr. `group_by()` partitions the data by one or more columns. `summarize()` computes aggregate statistics (mean, sum, count, etc.) for each group.

```
flights_tbl %>%
  group_by(carrier) %>%
  summarize(
    count = n(),
    avg_dep_delay = mean(dep_delay, na.rm = TRUE),
    avg_arr_delay = mean(arr_delay, na.rm = TRUE)
  ) %>%
  arrange(desc(count)) %>%
  collect()
```

```
## # A tibble: 16 x 4
##    carrier count avg_dep_delay avg_arr_delay
##    <chr>   <dbl>         <dbl>         <dbl>
##  1 UA      58665         12.1          3.56
##  2 B6      54635         13.0          9.46
```

```
##  3 EV      54173         20.0         15.8
##  4 DL      48110          9.26         1.64
##  5 AA      32729          8.59         0.364
##  6 MQ      26397         10.6         10.8
##  7 US      20536          3.78         2.13
##  8 9E      18460         16.7          7.38
##  9 WN      12275         17.7          9.65
## 10 VX       5162         12.9          1.76
## 11 FL       3260         18.7         20.1
## 12 AS        714          5.80        -9.93
## 13 F9        685         20.2         21.9
## 14 YV        601         19.0         15.6
## 15 HA        342          4.90        -6.92
## 16 OO         32         12.6         11.9
```

9 Duplicate Elimination Verbs (δ)

The relational algebra duplicate elimination operator δ corresponds to `distinct()` in dplyr. `distinct()` removes duplicate rows, optionally keeping only specified columns. Without arguments, it removes completely identical rows. With column arguments, it keeps only unique combinations of those columns.

```
flights_tbl %>%
  distinct(origin, dest) %>%
  arrange(origin, dest) %>%
  head(10) %>%
  collect()
```

```
## # A tibble: 10 x 2
##    origin dest
##    <chr>  <chr>
##  1 EWR    ALB
##  2 EWR    ANC
##  3 EWR    ATL
##  4 EWR    AUS
##  5 EWR    AVL
##  6 EWR    BDL
##  7 EWR    BNA
##  8 EWR    BOS
##  9 EWR    BQN
## 10 EWR    BTV
```

10 Sorting Verbs (τ)

The relational algebra sorting operator τ corresponds to `arrange()` in dplyr. `arrange()` orders rows by one or more columns in ascending order by default. Use `desc()` to sort in descending order. Multiple columns create hierarchical sorting (first by column 1, then by column 2, etc.)

```
flights_tbl %>%
  select(year, month, day, carrier, dep_delay, arr_delay) %>%
  arrange(desc(dep_delay)) %>%
```

```
  head(10) %>%
  collect()
```

```
## # A tibble: 10 x 6
##     year month   day carrier dep_delay arr_delay
##    <int> <int> <int> <chr>       <dbl>     <dbl>
##  1  2013     1     9 HA           1301      1272
##  2  2013     6    15 MQ           1137      1127
##  3  2013     1    10 MQ           1126      1109
##  4  2013     9    20 AA           1014      1007
##  5  2013     7    22 MQ           1005       989
##  6  2013     4    10 DL            960       931
##  7  2013     3    17 DL            911       915
##  8  2013     6    27 DL            899       850
##  9  2013     7    22 DL            898       895
## 10  2013    12     5 AA            896       878
```